



**UNIVERSIDADE FEDERAL DA BAHIA
INSTITUTO DE COMPUTAÇÃO
SISTEMAS DE INFORMAÇÃO**

**ARTIGO - SEMINÁRIO 2
DOMAIN DRIVEN DESIGN TÁTICO**

**Bruno Santos Fernandes
Edson Cerqueira Junior
Gabriel Lima Brito
Guilherme de Santana Soares Xavier
Jacques Barreto Salah
Lincoln Pereira da Silva
Néphi Santos Luduvico Andrade
Pedro Day Santana França
Samuel Marques Rodrigues de Miranda**

**Salvador/BA
2026**

Bruno Santos Fernandes
Edson Cerqueira Junior
Gabriel Lima Brito
Guilherme de Santana Soares Xavier
Jacques Barreto Salah
Lincoln Pereira da Silva
Néphi Santos Luduvico Andrade
Pedro Day Santana França
Samuel Marques Rodrigues de Miranda

ARTIGO - SEMINÁRIO 2
DOMAIN DRIVEN DESIGN TÁTICO

Trabalho apresentado à disciplina de
Programação Orientada a Objetos como
requisito de avaliação do Seminário 2.

Professor(a): Gilberto de Souza Leite.

SUMÁRIO

1	INTRODUÇÃO	4
2	DOMAIN-DRIVEN DESIGN	5
2.1	DDD Estratégico	5
2.1.1	Linguagem Ubíqua	6
2.1.2	Subdomínios	7
2.1.3	<i>Bounded Context</i>	8
2.1.4	<i>Context Map</i>	8
2.2	DDD Tático	9
3	ENTIDADES	10
3.1	Código Exemplo (Entidade Cliente)	10
4	VALUE OBJECTS	12
4.1	Imutabilidade	12
4.2	Igualdade por Valor	12
4.3	Ausência de Identidade	13
4.4	Auto-validação de Regras Locais	13
4.5	Diferença entre Entidade e <i>Value Objects</i>	14
5	DOMAIN SERVICES	16
5.1	O Uso de Domain Services	16
5.1.1	Execução de um Processo de Negócio Significativo	16
5.1.2	Transformação ou Conversão entre Objetos de Domínio	16
5.1.3	Cálculo de Valores com Múltiplos Objetos	17
5.1.4	Absorção da Coordenação	17
5.2	Domain Services e Modelo Anêmico	17
5.2.1	O uso de Services como "Bala de Prata"	18
5.2.2	Centralização da Lógica nos <i>Services</i>	18
5.2.3	A Criação de "Mini-camadas" de <i>Services</i>	18
6	AGGREGATES	19
6.1	Relações de domínio e limites de consistência	19
6.2	Exemplo prático da definição de agregados	19
6.3	Invariantes e regras de fronteira	19
6.4	<i>Aggregate Roots</i>	20

	6.5	Validação e garantia de consistência.....	20
7		FACTORIES E REPOSITORIES.....	21
	7.1	<i>Factories</i>	21
	7.2	<i>Repositories</i>	21
8		DDD BUROCRÁTICO.....	23
	8.1	Complexidade.....	23
	8.2	Verbosidade.....	23
	8.3	“Over-Engineering” em Cruds.....	23
	8.4	Dependência de Especialistas do Domínio.....	23
	8.5	Impacto na Performance.....	24
9		CONCLUSÃO.....	25
		REFERÊNCIAS BIBLIOGRÁFICAS.....	26

1 INTRODUÇÃO

O desenvolvimento de sistemas modernos enfrenta o desafio constante de alinhar a implementação técnica às complexas necessidades de negócio. O *domain-driven design* (DDD), proposto por Eric Evans em 2003, surge como uma metodologia voltada a reduzir essa complexidade por meio de uma modelagem expressiva e colaborativa. O objetivo central dessa abordagem é estabelecer uma conexão fiel entre o código e o domínio do negócio, evitando que o *software* se torne desconexo de seu propósito original ou de difícil manutenção.

A aplicação do DDD é fundamentada em dois pilares: o estratégico e o tático. Enquanto a modelagem estratégica organiza o domínio em fronteiras e relações claras, o DDD Tático foca na implementação técnica interna, utilizando padrões para criar um modelo rico e estritamente alinhado à linguagem ubíqua. Este artigo explora os principais blocos de construção táticos — como entidades, *value objects*, agregados, *factories* e *repositories* — e discute os riscos do "DDD Burocrático", que ocorre quando a metodologia é aplicada sem uma necessidade real, resultando em complexidade desnecessária.

2 DOMAIN-DRIVEN DESIGN

Domain-driven design (DDD) é uma abordagem de desenvolvimento de software proposta por Eric Evans no livro *"Domain-Driven Design: Tackling Complexity in the Heart of Software"* publicado em 2003. *Domain-Driven Design* não é um *framework*, muito menos uma linguagem de programação (e nem mesmo está atrelada a uma linguagem específica). O DDD é uma metodologia que tem como objetivo fazer uma **conexão entre o código e o domínio do negócio**.

Essa técnica surge como uma maneira de reduzir a complexidade de um projeto por meio de uma modelagem expressiva e colaborativa. Para isso, os times precisam modelar um projeto baseado fortemente na comunicação usando principalmente a **linguagem ubíqua**. O DDD é visto como uma solução para a dificuldade de traduzir as reais necessidades do negócio para o *software*. A não utilização dessa metodologia pode resultar em sistemas de difícil manutenção e desconexos com o objetivo final da empresa.

Para aplicar o DDD, é necessário ter um bom entendimento do objeto principal do negócio: **o domínio**. Domínio é o problema que o *software* se propõe a resolver. Por exemplo, em um sistema bancário, o domínio seria a gestão financeira; já em um hospital, seria o atendimento e a gestão de pacientes. Os responsáveis por esclarecer as nuances do domínio para os desenvolvedores são os chamados *"domain experts"* (Especialistas de Domínio). O DDD propõe que o *software* deve refletir fielmente esse domínio, usando a mesma linguagem e os mesmos conceitos do negócio.

O *domain-driven design* é apoiado em dois pilares principais: **o estratégico e o tático**. Para aplicar o DDD Tático, não é estritamente necessário passar pelo DDD Estratégico, mas é extremamente recomendado e considera-se uma boa prática.

2.1 DDD Estratégico

A modelagem estratégica é a primeira camada do *Domain-Driven Design*. Para realizar o DDD estratégico, é necessário ter uma visão panorâmica do negócio, pois, nessa etapa, o objetivo é organizar e dividir o domínio em partes gerenciáveis, definindo fronteiras claras e relações entre equipes e modelos. A modelagem

estratégica é o processo que tem como propósito estruturar como o DDD tático irá atuar no desenvolvimento do software.

2.1.1 Linguagem Ubíqua

Para entendermos como relacionar os conceitos de DDD a um contexto de desenvolvimento de sistemas é necessário que, tanto a parte interessada (isto é, os *stakeholders*), quanto o time de desenvolvimento, tenham uma linguagem em comum para explicitar seus interesses e evitar ruídos na comunicação. É a partir daí que surge a linguagem ubíqua, uma forma de linguagem que relaciona a linguagem do cliente com a do time de desenvolvimento.

Como exemplo podemos considerar o contexto de um sistema para um gabinete jurídico. Temos como clientes advogados, juízes, desembargadores e muitos outros cargos que são especialistas no domínio. Estes, por sua vez, conhecem e utilizam terminologias e jargões próprios das atividades que exercem no contexto. Para um time de desenvolvimento é de extrema importância entender o que está sendo falado, e de forma análoga, um time de desenvolvedores também têm sua própria linguagem e jargões técnicos do seu contexto de atuação. Sem um alinhamento dessas duas linguagens podem surgir falhas de entendimento e comprometer o sistema severamente.

Portanto, para solucionar o problema, as duas partes se juntam para formar uma forma de linguagem que será utilizada em reuniões e documentos, a **linguagem ubíqua**, justamente com a finalidade de evitar ambiguidades.

A **modelagem orientada a objetos** será o meio entre a Linguagem Ubíqua e a linguagem de programação para o sistema, uma vez que esta já detém ferramentas que possibilitam a criação dos contextos do mundo real, isto é, domínios e subdomínios por meio de classes e métodos.

Desta forma, a Linguagem Ubíqua determina:

- os nomes dos elementos do domínio;
- a definição das entidades;
- as responsabilidades de cada objeto;
- sua relação dos elementos do sistema com seus subdomínios.

2.1.2 Subdomínios

No DDD, um subdomínio representa uma área menor e bem definida de conhecimento ou responsabilidade dentro de um domínio maior e, se preciso, um subdomínio pode ser dividido em subdomínios ainda menores. Há diversos benefícios em dividir um domínio em partes menores. Alguns desses benefícios são:

- melhor compreensão e redução de complexidade;
- desenvolvimento e implantação independentes;
- promoção de uma melhor comunicação;
- estruturação para organizar e gerenciar o domínio.

Há três tipos de subdomínio: ***core domain***, ***supporting subdomain*** e ***generic subdomain***.

O *core domain* (ou domínio central) é o diferencial competitivo do negócio. Ele representa o coração do negócio, onde reside a lógica mais complexa, o maior valor estratégico e a principal motivação da aplicação. Esse domínio merece o máximo de investimento do time. Diferentemente dos outros subdomínios, o core domain **deve ser desenvolvido internamente**. Um exemplo de domínio central: o motor de recomendação de uma plataforma de *streaming*.

O *supporting subdomain* (ou subdomínio de suporte) é responsável por apoiar o subdomínio central, mas não é o diferencial da empresa. Ele fornece funções auxiliares para o *core domain*. O subdomínio de suporte geralmente envolve serviços compartilhados, utilitários ou funcionalidades comuns que várias partes do projeto podem aproveitar. Pode ser desenvolvido internamente com menos rigor. Um exemplo de subdomínio de suporte: um sistema de cadastro de usuários.

O *generic subdomain* (ou subdomínio genérico) pode ser classificado como um problema genérico, sem diferencial. Um subdomínio genérico é uma área do negócio que é importante para as operações, mas não oferece uma vantagem competitiva. Muitas vezes, é mais vantajoso utilizar ou comprar uma solução pronta pois muitos times gastam energia no lugar errado, **tratando subdomínios genéricos como se fossem core**. Um exemplo de subdomínio genérico: um sistema de autenticação e autorização.

2.1.3 Bounded Context

O *bounded context* (contexto delimitado) é o **conceito central** do DDD Estratégico. Ele pode ser definido como uma fronteira que delimita explicitamente a responsabilidade de cada parte do sistema. Dentro dessa fronteira, existe um modelo consistente, com sua própria abordagem de arquitetura e sua própria linguagem ubíqua. Uma premissa fundamental é que, fora dessa fronteira, um mesmo termo pode assumir significados e regras de negócio completamente diferentes.

Por exemplo: um "produto" no contexto de catálogo possui atributos como nome, descrição e fotos. Já no contexto de estoque, esse mesmo "produto" é definido por sua quantidade em prateleira e localização no armazém. Embora representem a mesma entidade no mundo real, eles são modelos de código distintos. Tentar unificá-los em um único modelo resulta em classes excessivamente complexas, frágeis e de difícil manutenção.

2.1.4 Context Map

O *context map* (mapa de contexto) documenta como os *bounded contexts* se relacionam entre si. Trata-se de uma representação estratégica que mapeia como diferentes partes de um domínio se comunicam e se integram. Existem diversos padrões de relacionamento:

- **Shared Kernel:** Dois contextos compartilham um subconjunto do modelo (como uma biblioteca comum ou banco de dados). Mudanças nesse núcleo requerem coordenação estreita entre os times.
- **Customer-Supplier:** Um relacionamento de "Cliente-Fornecedor" onde um contexto (*upstream*) fornece dados ou serviços para outro (*downstream*). O *downstream* depende do *upstream*.
- **Conformist:** O contexto *downstream* adota integralmente o modelo do *upstream* sem realizar adaptações, aceitando os termos e a estrutura fornecidos.

- **Anti-Corruption Layer (ACL):** O *downstream* cria uma camada de tradução (isolamento) para se proteger de modelos externos, garantindo que sua lógica interna permaneça limpa.
- **Open Host Service:** O *upstream* publica uma API ou serviço bem definido e documentado para múltiplos consumidores, facilitando a integração.
- **Published Language:** Utiliza uma linguagem de integração comum e bem documentada para a troca de informações (ex: JSON Schema, Protobuf ou XML).
- **Separate Ways:** Os contextos não possuem integração técnica. Cada um resolve seu problema de forma totalmente independente.
- **Big Ball of Mud:** Identifica um contexto legado ou mal estruturado, sem fronteiras claras. Geralmente, deve ser isolado por uma ACL para não comprometer novos desenvolvimentos.

2.2 DDD Tático

A modelagem tática representa a implementação técnica interna de um *bounded context*. Nessa etapa, utiliza-se um conjunto de padrões e práticas para transpor o domínio para o código, criando um modelo rico, expressivo e estritamente alinhado à linguagem ubíqua. Trata-se do momento em que todo o planejamento realizado no DDD Estratégico é traduzido em estruturas de software fundamentadas em programação orientada a objetos.

3 ENTIDADES

Em DDD, as entidades representam os objetos do domínio que possuem identidade própria, sendo distinguíveis independente de seus atributos e cumprindo regras para serem consideradas entidades, sendo elas:

- possuir ciclo de vida contínuo mantendo a sua identidade;
- ser capaz de mudar de estado;
- ter identificador Único que define sua identidade;

Estes atributos são essenciais para que as entidades consigam ser manipuladas ao longo de todo o ciclo de vida do *software*, garantido sua implementação de forma correta dentro das regras de negócio que o sistema necessita.

3.1 Código Exemplo (Entidade Cliente)

```
public class Cliente {  
  
    private final String cpf;  
    private String nome;  
    private String endereco;  
    private String telefone;  
    private String email;  
  
    public Cliente(String cpf, String nome, String endereco, String  
telefone, String email) {  
        if(cpf == null || cpf.isBlank()){  
            throw new IllegalArgumentException("CPF inválido!");  
        }  
        if(nome == null || nome.isBlank()){  
            throw new IllegalArgumentException("Nome inválido!");  
        }  
        this.cpf = cpf;  
        this.nome = nome;  
        this.endereco = endereco;  
        this.telefone = telefone;  
        setEmail(email);  
    }  
  
    public void atualizarEndereco(String novoEndereco) {  
        if (novoEndereco == null || novoEndereco.isBlank()) {  
            throw new IllegalArgumentException("Endereço inválido");  
        }  
    }  
}
```

```

        this.endereco = novoEndereco;
    }

    public void atualizarTelefone(String telefone) {
        this.telefone = telefone;
    }

    public void setEmail(String email) {
        if (email == null || !email.contains("@")) {
            throw new IllegalArgumentException("Email inválido");
        }
        this.email = email;
    }

    public String getCpf() {
        return cpf;
    }
}

```

O código acima ilustra uma entidade Cliente, onde ela incorpora como regra de negócio:

- validar CPF;
- impedir email inválido;
- atualizar telefone com regra;
- não permitir nem nome nem CPF vazios.

4 VALUE OBJECTS: CONCEITOS E FUNDAMENTOS

No contexto do **domain-driven design (DDD)**, os *value objects* representam um dos blocos fundamentais para a modelagem de domínios ricos e expressivos. Diferentemente das entidades, cujo foco está na identidade e no ciclo de vida, os *value objects* são definidos exclusivamente pelos seus atributos e pelo significado que carregam dentro do domínio. Este tópico aborda suas principais características: imutabilidade, igualdade por valor, ausência de identidade e auto-validação de regras locais.

4.1 Imutabilidade

A imutabilidade é uma das propriedades centrais dos *value objects*. Um objeto imutável é aquele cujo estado não pode ser alterado após sua criação. Isso significa que qualquer modificação necessária resulta na criação de uma nova instância, ao invés da alteração da instância existente.

Essa característica traz benefícios importantes para o design de *software*. Primeiramente, promover maior previsibilidade no comportamento do sistema, já que objetos imutáveis não sofrem alterações inesperadas ao longo do tempo. Além disso, facilita a concorrência, pois elimina problemas relacionados a condições de corrida (*race conditions*), uma vez que múltiplas threads podem acessar o mesmo objeto sem risco de inconsistência.

No contexto do DDD, a imutabilidade reforça a ideia de que *value objects* representa conceitos estáticos do domínio, como um endereço, uma data ou um valor monetário. Por exemplo, ao alterar o valor de um objeto que representa dinheiro, não se modifica o objeto original, mas sim cria-se um novo com o valor atualizado.

4.2 Igualdade por Valor

Outra característica essencial dos *value objects* é a igualdade baseada em valor. Diferentemente das entidades, que são comparadas por identidade (por exemplo, um identificador único), os *value objects* são considerados iguais quando todos os seus atributos possuem valores equivalentes. Essa abordagem reflete

diretamente a natureza desses objetos no domínio. Por exemplo, dois objetos que representam a mesma data (dia, mês e ano iguais) devem ser considerados equivalentes, independentemente de serem instâncias distintas na memória.

A implementação dessa característica exige a sobrescrita adequada dos métodos de comparação, como “*equals()*” e “*hashCode()*” em linguagens orientadas a objetos como Java. Essa prática garante consistência em estruturas de dados e operações que dependem de comparação, como coleções. A igualdade por valor também contribui para um design mais expressivo e alinhado com o domínio, pois elimina a necessidade de rastrear identidades desnecessárias, simplificando a modelagem.

4.3 Ausência de Identidade

Os *value objects* não possuem identidade própria. Isso significa que não existe um atributo único que os diferencie de outras instâncias com os mesmos valores. Sua existência é completamente definida pelos dados que carregam. Essa ausência de identidade implica que não há necessidade de gerenciamento de ciclo de vida, como ocorre com entidades. Não é necessário persistir ou rastrear um *value object* individualmente ao longo do tempo. Em vez disso, eles são frequentemente utilizados como componentes de entidades ou agregados.

Por exemplo, um endereço dentro de um sistema pode ser modelado como um *value object*. Se dois endereços possuem os mesmos atributos (rua, número, cidade, etc.), eles são indistinguíveis do ponto de vista do domínio, mesmo que existam como objetos separados na aplicação. Essa característica reduz a complexidade do sistema, pois evita a introdução de identificadores artificiais e mantém o foco no significado dos dados.

4.4 Auto-validação de regras locais

Os *value objects* são responsáveis por garantir sua própria consistência interna. Isso significa que todas as regras de validação relacionadas aos seus atributos devem ser aplicadas no momento de sua criação. Esse princípio é conhecido como auto-validação e assegura que um *value object* nunca exista em um estado inválido. Por exemplo, um objeto que representa um CPF deve garantir que o

valor fornecido seja válido de acordo com as regras de verificação desse documento. Caso contrário, a criação do objeto deve ser impedida.

A auto-validação contribui para a robustez do sistema, pois evita a propagação de dados inconsistentes. Além disso, centraliza as regras de negócio locais dentro do próprio objeto, promovendo coesão e reduzindo a duplicação de lógica em diferentes partes do sistema. Essa abordagem também facilita a manutenção e evolução do código, uma vez que alterações nas regras de validação precisam ser feitas em apenas um local.

4.5 Diferença entre Entidade e *Value Objects*

No *domain-driven design*, um *value object* (objeto de valor) é um elemento do modelo de domínio que representa um conceito descritivo, mensurável ou observável do negócio. Ao contrário das entidades, os *value objects* não possuem identidade própria: dois objetos de valor são considerados iguais se, e somente se, todos os seus atributos forem iguais. Essa característica é chamada de igualdade estrutural ou igualdade por valor. Já a igualdade entre entidades é definida pelo identificador: dois objetos “Funcionário” com o mesmo ID são o mesmo funcionário, independentemente de qualquer outro atributo.

Além da ausência de identidade, os *value objects* são, por definição, imutáveis. Uma vez criado, um objeto de valor não pode ter seus atributos alterados. Quando é necessário representar uma nova versão daquele conceito, o objeto anterior é simplesmente descartado e substituído por um novo, em vez de ser modificado. Essa imutabilidade traz benefícios diretos para a segurança e a previsibilidade do modelo, eliminando efeitos colaterais indesejados causados por alterações de estado.

Exemplos clássicos de *value objects* incluem: endereços postais (rua, número, cidade, CEP), períodos de tempo (data de início e data de fim), coordenadas geográficas, quantias monetárias (valor e moeda) e intervalos numéricos. Nenhum desses conceitos precisa de um identificador único para existir; o que os define inteiramente é a combinação de seus atributos.

Outra distinção diz respeito ao ciclo de vida. Entidades possuem um ciclo de vida explícito: nascem, evoluem e podem ser inativadas ou removidas. Um Pedido passa pelos estados de aberto, confirmado, enviado e entregue. *Value objects*, por

sua vez, não possuem ciclo de vida próprio: eles simplesmente existem enquanto são necessários ao objeto que os contém e deixam de existir quando são substituídos ou quando a entidade que os referencia é removida.

Tabela 1 - Entidade e Value Object

Características	Entidade	Value Object
Identidade:	Possui identificador único	Definido por seus atributos
Mutabilidade:	Mutável	Imutável
Igualdade:	Por identidade (ID)	Por valor (atributos)
Ciclo de vida:	Explícito e contínuo	Inexistente - é substituído
Exemplos:	Cliente, Pedido, Produto	Endereço, Dinheiro, Email

Fonte: Elaborado pelos autores (2026).

5 DOMAIN SERVICES

No contexto do *Domain-Driven Design* (DDD), um Domain Service representa uma operação sem estado (*stateless*) responsável por encapsular uma tarefa específica e significativa do domínio. Diferentemente das Entidades (que carregam identidade e ciclo de vida) e dos Objetos de Valor (que modelam conceitos descritivos e imutáveis), o Domain Service existe precisamente para abrigar comportamentos que não pertencem naturalmente a nenhum objeto isolado.

O principal indicador da necessidade de um Domain Service é o chamado "desconforto de alocação", isto é, quando o desenvolvedor percebe que determinada operação poderia ser colocada em uma Entidade, mas essa escolha pareceria forçada ou violaria o princípio de responsabilidade única, está diante de um forte sinal de que o Service é a abstração correta. Evans (2003) descreve esse sintoma como a operação que "se sente deslocada" (*feels out of place*) em qualquer objeto do modelo.

5.1 O Uso de Domain Services

A aplicação adequada de **domain services** pode ser organizada em torno de três cenários canônicos:

5.1.1 Execução de um processo de negócio significativo

Quando uma operação encapsula uma ação relevante do domínio que transcende as responsabilidades de um único objeto, o *domain service* torna-se a escolha natural. A operação existe no léxico do domínio, ou seja, ela tem nome, tem regras, tem impacto, mas não é propriedade conceitual de nenhuma Entidade específica. Logo, modelá-la como método de uma entidade arbitrária comprometeria a coesão do modelo.

5.1.2 Transformação ou conversão entre objetos de domínio

Situações que envolvem a conversão de um objeto de domínio de uma composição para outra, por exemplo, transformar um "Pedido" em uma "NotaFiscal"

frequentemente exigem lógica que conhece ambas as partes, sem pertencer a nenhuma delas. Um *domain service* garante que essa lógica de conversão fique expressa no domínio, e não vazada para camadas superiores da aplicação.

5.1.3 Cálculo de valores com múltiplos objetos

Este é talvez o cenário mais recorrente. Quando um cálculo requer a entrada de duas ou mais Entidades ou Objetos de Valor, atribuí-lo a qualquer uma delas cria um acoplamento artificial e confuso. O *domain service* atua como o local natural para essa coordenação.

Exemplo canônico: O cálculo das prioridades de negócio de um produto exige a recuperação de todos os “*BacklogItems*” pendentes e a iteração sobre as avaliações de risco, custo e benefício de cada um deles para produzir um objeto de valor final (“*BusinessPriorityTotals*”). Nenhum “*BacklogItem*” individualmente possui visibilidade suficiente para realizar esse cálculo; tampouco o Produto deveria conhecer os detalhes internos de avaliação de cada item. O serviço é, portanto, o único lugar onde essa colaboração pode ocorrer sem violar o encapsulamento.

5.1.4 Absorção da coordenação

Um Domain Service também cumpre papel fundamental ao proteger o cliente de precisar compreender as regras do negócio para orquestrar um fluxo. Quando a lógica de coordenação entre objetos precisa existir em algum lugar, o *domain service* garante que ela permaneça na camada de domínio, e não migre indevidamente para fora dela.

Exemplo canônico: A autenticação de um usuário envolve verificar se o Tenant está ativo, criptografar a senha submetida para compará-la com a armazenada e confirmar que o *user* está habilitado. Distribuir essas verificações entre as entidades individuais criaria um acoplamento indesejado; delegá-las ao cliente violaria o encapsulamento do domínio. Um *AuthenticationService* absorve toda essa coordenação, mantendo as regras onde devem estar.

5.2 Domain Services e Modelo Anêmico

O Modelo Anêmico ocorre quando as classes de domínio são reduzidas a meros contêineres de dados (*getters* e *setters*), enquanto toda a lógica de negócio migra para serviços externos ao objeto. O resultado é uma inversão estrutural: o modelo passa a parecer procedural disfarçado de orientado a objetos, desperdiçando os benefícios fundamentais do paradigma: encapsulamento, coesão e expressividade comportamental.

5.2.1 O Uso de *Services* como "Bala de Prata"

O primeiro sinal de alerta é a proliferação indiscriminada de *domain services*. Quando qualquer conceito do domínio é modelado como *service* por comodidade ou por hábito, as entidades e objetos de valor são gradualmente esvaziados de seus comportamentos naturais. Uma conta bancária que não sabe realizar um “saque()”, delegando essa responsabilidade a um conta bancária *service*, é o exemplo paradigmático desse problema: o objeto existe, mas não age.

5.2.2 Centralização da Lógica nos *Services*

O anti-padrão se consolida quando toda a lógica de domínio passa a residir nos *services*, em vez de estar distribuída pelos comportamentos das Entidades e objetos de Valor. Nesse cenário, os *services* acumulam responsabilidades que deveriam ser intrínsecas aos objetos, e o modelo perde sua capacidade de comunicar intenção. O código resultante tende a ser mais difícil de manter, testar e evoluir, pois a lógica não está onde o leitor do modelo esperaria encontrá-la.

5.2.3 A criação de "Mini-camadas" de *Services*

Um terceiro padrão recorrente é a tentativa de criar uma camada uniforme de *Domain Services* posicionada acima de todo o restante do modelo de domínio. Essa abordagem, frequentemente motivada por uma leitura superficial do DDD ou por vícios de designs transacionais, conduz inevitavelmente ao Modelo Anêmico, pois pressupõe que os objetos do domínio são passivos e que toda ação de relevância deve passar por um *service* intermediário.

6 AGGREGATES

Segundo conceitos definidos por Evans (2003), os agregados (ou *aggregates*) surgem da necessidade de definir limites de consistência nas relações entre entidades e objetos de valor.

6.1 Relações de domínio e limites de consistência

Cada relação entre os elementos de um *software* é regida pelas regras de domínio, que possuem diferentes limites de consistência para cada tipo de relação. Um exemplo dessa ideia são as relações de associação (quando um elemento apenas refere-se a outro) e composição (quando um elemento é parte essencial de outro). Neste sentido, os agregados agem como um limite de consistência que envolve somente os elementos ligados pelo mesmo conjunto.

6.2 Exemplo prático da definição de agregados

Imagine um *software* em que temos as entidades “Cliente” e “Pedido”, e o objeto “Endereço”. Qual o limite no relacionamento entre tais elementos? A ideia de agregados surge, justamente, para evitar que a consistência do software seja afetada pela má manutenção dessas relações. O cliente deve existir ainda que não exista o pedido, logo, a relação “Cliente-Pedido” é uma associação. No entanto, em caso de um negócio que funcione apenas por meio de entregas, não há possibilidade de existir um pedido sem endereço, portanto, o endereço pode ser definido como elemento essencial ao pedido, tornando a relação “Pedido-Endereço” em uma composição. Neste exemplo, seria possível definir dois agregados: um que envolve o elemento Cliente e outro que envolve o elemento “Pedidos” (incluindo o “Endereço”) ligado ao agregado do cliente por uma relação de referência.

6.3 Invariantes e regras de fronteira

Nesse contexto, os agregados também são responsáveis por garantir os invariantes do domínio, ou seja, regras que devem permanecer sempre válidas entre os elementos que compõem o conjunto. Para isso, as mudanças nesses elementos

devem ocorrer de forma controlada dentro dos limites do agregado, evitando alterações diretas que possam comprometer sua consistência. Além disso, as regras de fronteira definem que a consistência deve ser mantida apenas entre os elementos internos do agregado, enquanto as relações com outros agregados ocorrem por referência.

6.4 Aggregate Roots

Outro conceito importante dos agregados de Evans são as raízes (ou roots). A raiz de um agregado é, conceitualmente, o elemento que pode ser acessado por elementos externos ao agregado, enquanto os outros elementos apenas podem ser acessados e alterados através do acesso à raiz. Tal conceito impede que alterações diretas em objetos e entidades essenciais comprometam a consistência do agregado.

6.5 Validação e garantia de consistência

Além disso, Evans (2003) define o limite de consistência, que pode ser analisado como um conjunto de elementos que deve permanecer válido após mudanças em um objeto do conjunto. Para validação, a mudança é confirmada na raiz do agregado, que coordena a mudança nos outros elementos do limite. Esse mecanismo evita o estado inválido de objetos e entidades e a violação das regras de negócio, garantindo a consistência entre os elementos de um agregado.

7 **FACTORIES E REPOSITORIES**

À medida que o projeto aumenta em complexidade, vai tornando-se igualmente trabalhosa e densa a gerência de seus componentes e do seu respectivo ciclo de vida. Nesse aspecto, o DDD propõe os conceitos de *factories* e *repositories* (EVANS, 2003) para melhor gerenciar a criação e armazenamento, respectivamente, de entidades e agregados.

7.1 **Factories**

Factories são componentes responsáveis por lidar com a criação de objetos de domínio complexos. O aumento da complexidade manifesta-se em situações como: alta quantidade de validações necessárias, utilização de diversos objetos distintos na criação, presença de regras de negócio no processo de criação e lógica de fabricação dispersa pelo sistema.

A ideia central por trás das *factories* é a separação entre criação e comportamento. Não é vantajoso atribuir a um objeto complexo a responsabilidade de gerenciar sua própria criação, tendo em mente todas as responsabilidades que já possui referentes ao seu comportamento. Em analogia, a *factory* seria um robô que monta motores de carro, enquanto que um objeto sem *factory* correspondente seria um motor que se autoconstroi. Nesse aspecto, não é viável elaborar um motor que monta a si mesmo, nem é responsabilidade do motor se autoconstruir. Ademais, a *factory* possui o benefício de centralizar e encapsular a lógica de criação, fazendo com que eventuais alterações sejam feitas em um único local e aumentando a organização e a segurança do código. Além disso, garante o respeito às invariantes do domínio no momento da criação.

É importante frisar que recomenda-se usar *factories* apenas para objetos complexos. Em casos onde um construtor simples é suficiente (por não haver regras de negócio ou validações significativas, por exemplo) não há necessidade de as utilizar sob risco de incidir em *over-engineering*.

7.2 **Repositories**

Enquanto as *factories* lidam com o início da vida dos objetos, os *repositories* são responsáveis pelo meio e fim desse ciclo (persistência e recuperação, principalmente com raízes de agregados). O *repository* é definido como uma interface na camada de domínio, enquanto sua implementação pertence à camada de infraestrutura. Eles possuem como função principal salvar e buscar objetos no mecanismo de armazenamento (como um banco de dados). No contexto do domínio, eles se comportam como uma coleção de objetos em memória. Mas na verdade, implementam uma comunicação com o banco de dados sem que o domínio o acesse diretamente. Cabe ressaltar que *repositories* não devem conter regras de negócio, se restringindo à persistência e à recuperação de dados.

Idealmente, o domínio não deve acessar o banco de dados diretamente. A utilidade do *repository* vem ao ser uma camada de abstração entre o domínio e o armazenamento, promovendo desacoplamento. Assim, o domínio permanece independente da tecnologia usada para armazenar dados, e eventuais alterações nesta não implicam em alterações no domínio. Ou seja, se o banco de dados é alterado de SQL para MongoDB, por exemplo, deverá se alterar apenas a implementação do *repository*. Ademais, tal abordagem facilita a testagem do sistema, pois permite o uso de implementações em memória ou de mocks de bancos de dados.

8 DDD BUROCRÁTICO

Como visto neste artigo, a implementação do DDD em aplicações pode trazer diversos benefícios, como facilitar a comunicação entre os membros da equipe através de uma linguagem ubíqua e a modularização do código em contextos delimitados. Porém o uso burocrático do *Domain-Driven Design*, isto é, utilizá-lo sem uma necessidade real de negócio, pode trazer mais estresse aos desenvolvedores do que vantagens.

8.1 Complexidade

O DDD introduz uma série de conceitos abstratos e padrões táticos (Entidades, Objetos de valor, Agregados, Repositórios, Fábricas, etc.), construindo de tal modo uma barreira de entrada elevada que exige uma equipe com conhecimentos profundos de arquitetura para construir um DDD eficaz.

8.2 Verbosidade

“*Overhead* de desenvolvimento”, ou simplesmente verbosidade, ocorre frequentemente quando o DDD é usado em aplicações mais simples. O tempo para modelar o domínio e definir a linguagem ubíqua retarda o início do desenvolvimento em si, o que pode colocar startups iniciantes em risco, já que o tempo é essencial para essas organizações.

8.3 “*Over-Engineering*” Em Cruds

Se a aplicação for somente um CRUD, ou seja, possui somente entradas e saídas com poucas lógicas de negócio, a utilização do DDD tornaria um código potencialmente simples em um código complexo e desnecessariamente verboso. A manutenção deste código também se tornaria mais difícil pois os desenvolvedores teriam de navegar por mais camadas para alterar pontos específicos.

8.4 Dependência de Especialistas do Domínio

O sucesso do DDD depende da colaboração estreita dos desenvolvedores e dos “*domain experts*” (pessoas que entendem o negócio). Se os domain experts não estiverem disponíveis para a construção de uma linguagem ubíqua, os desenvolvedores terão que a criar partindo de suposições, invalidando o propósito do DDD.

8.5 Impacto na Performance

Para garantir a consistência, o DDD sugere carregar o "Agregado" completo. Em modelos mal desenhados, isso pode resultar em consultas pesadas e consumo excessivo de memória. Também há o fato de que o DDD em si promove a criação de códigos mais verbosos, o que pode custar milissegundos a mais de tempo de execução. Apesar de pouco, essa diferença pode ser o fator determinante para o sucesso ou fracasso de sistemas em que o gerenciamento de tempo é fundamental.

9 CONCLUSÃO

A adoção do *Domain-Driven Design* (DDD) é uma escolha estratégica para reduzir a complexidade técnica por meio de uma conexão profunda entre o código e o domínio de negócio. O sucesso dessa abordagem reside na harmonia entre a organização estratégica de Contextos Delimitados e a aplicação rigorosa de padrões táticos, como Entidades, Agregados e *Value Objects*.

No entanto, o conceito de "DDD Burocrático" serve como um alerta essencial: o uso indiscriminado da metodologia em aplicações simples ou CRUDs resulta em verbosidade e *over-engineering* que podem prejudicar a agilidade do desenvolvimento. Conclui-se que o DDD deve ser priorizado no *Core Domain*, exigindo discernimento para que a arquitetura atue como um reflexo fiel das necessidades do especialista de domínio e não como uma barreira técnica desnecessária.

Em suma, o sucesso na aplicação do DDD exige do arquiteto e dos desenvolvedores não apenas domínio técnico, mas discernimento crítico. Deve-se saber identificar o Core Domain para onde os esforços devem ser direcionados, ao mesmo tempo em que se evita a armadilha do over-engineering em partes menos críticas do sistema. Quando bem aplicado, o DDD não apenas resolve problemas de software, mas cria uma ponte sólida entre a intenção do negócio e a execução técnica, resultando em sistemas que são, de fato, reflexos fiéis das necessidades que se propõem a atender.

REFERÊNCIAS BIBLIOGRÁFICAS:

EVANS, Eric. *Domain-Driven Design: tackling complexity in the heart of software*. Boston: Addison-Wesley, 2003.

SILVA, Pedro Emanuel. **Entendendo modelagem estratégica e tática do Domain-Driven Design (DDD)**. *Medium*, set. 2025. Disponível em: <https://pedrosilvatech.medium.com/entendendo-modelagem-estrategica-e-tatica-do-domain-driven-design-ddd-8a3e41da89db>. Acesso em: 19 de abr. 2026.

MICROSOFT. **Use tactical DDD to design microservices**. *Azure Architecture Center*, fev. 2026. Disponível em: <https://learn.microsoft.com/pt-br/azure/architecture/microservices/model/tactical-domain-driven-design>. Acesso em: 19 de abr. 2026.

MICROSOFT. **Value object design**. Disponível em: <https://learn.microsoft.com/en-us/dotnet/architecture/microservices/microservice-ddd-cqrs-patterns/implement-value-objects>. Acesso em: 19 abr. 2026.

DevIQ. **Value Object**. Disponível em: <https://deviq.com/domain-driven-design/value-object/>. Acesso em: 20 abr. 2026.

Martin Fowler. **Value Object**. Disponível em: <https://martinfowler.com/bliki/ValueObject.html>. Acesso em: 20 abr. 2026.

BRITO, Bruno. **Domain-Driven Design: a fórmula mágica ou só mais uma sigla pra deixar seu projeto pelado?** *Avera Blog*, maio. 2019. Disponível em: <https://avera.com.br/blog/domain-driven-design/>. Acesso em: 19 abr. 2026.

SILVA, Thiago Henrique. **Domain Driven Design**. *GitHub Pages*, jul. 2020. Disponível em: <https://thiagosilva95.github.io/ddd/>. Acesso em: 19 abr. 2026.

DEVIQ. **Subdomain**. *DevIQ*, [s.d.]. Disponível em: <https://deviq.com/domain-driven-design/subdomain/>. Acesso em: 19 abr. 2026.

STACK OVERFLOW EM PORTUGUÊS. **O que são e como identificar subdomínios em DDD?** *Stack Overflow em Português*, [s.d.]. Disponível em: <https://pt.stackoverflow.com/questions/49904>. Acesso em: 19 abr. 2026.

CÓDIGO FONTE TV. **DDD (Domain-Driven Design) // Dicionário do Programador**. *YouTube*, 27 jul. 2020. Disponível em: <https://youtu.be/GE6asEjTFv8>. Acesso em: 19 abr. 2026.

MARSTON, TONY. **Why i don't do Domain Driven Design**. *Tony Marston's Blog*, jul 2018. Disponível em: <https://www.tonymarston.net/php-mysql/dont-do-domain-driven-design.html>. Acesso em 19 abr. 2026.

EVANS, Eric. **Domain-Driven Design: tackling complexity in the heart of software**. Boston: Addison-Wesley, 2003.

MICROSOFT. **Use tactical DDD to design microservices**. Azure Architecture Center, 2026. Disponível em: <https://learn.microsoft.com/pt-br/azure/architecture/microservices/model/tactical-domain-driven-design>. Acesso em: 19 abr. 2026.

VERNON, Vaughn. **Implementing Domain-Driven Design**. Upper Saddle River: Addison-Wesley, 2013.